베르나챠(Vernazza) 절벽 마을, 이탈리아

부록

1. 상속의 종류 : public, protected, private 상속
2. 가상 상속
3. auto와 람다식
4. 파일 입출력 스트림 상태 검사
5. 파일 입출력 임의 접근

1. 상속의 종류 : public, protected, private 상속 [8장 관련]

public 상속

상속을 통해 기본 클래스의 멤버들이 파생 클래스의 멤버로 확장될 때, 기본 클래스 멤버의 접근 지정은 상속 조건에 따라 달라진다. 지금까지는 다음과 같은 'public 상속'에 대해서만 다루었다.

```
class Derived : public Base {
    ...
};
```

private 상속
protected 상속

'public 상속'을 사용하면 Base에 선언된 멤버들은 접근 지정을 그대로 유지한 채 Derived의 멤버로 확장된다. 하지만, 'private 상속'이나 'protected 상속'은 기본 클래스에 선언된 멤버들의 접근 지정을 변경한다. [그림 1-1]은 public, protected, private으로 상속하는 경우, 기본 클래스의 멤버들이 파생 클래스의 멤버로 확장될 때 접근 지정의 변화를 보여준다.

● public 상속

기본 클래스를 public으로 상속받으면, 기본 클래스의 protected, public 멤버들은 접근 지정 변경 없이 파생 클래스에 그대로 상속 확장된다.

● protected 상속

기본 클래스를 protected로 상속받으면, 기본 클래스의 protected, public 멤버들은 모두 protected 접근 지정으로 변경되어 파생 클래스에 상속 확장된다.

● private 상속

기본 클래스를 private으로 상속받으면, 기본 클래스의 protected, public 멤버들은 모두 private 접근 지정으로 변경되어 파생 클래스에 상속 확장된다.

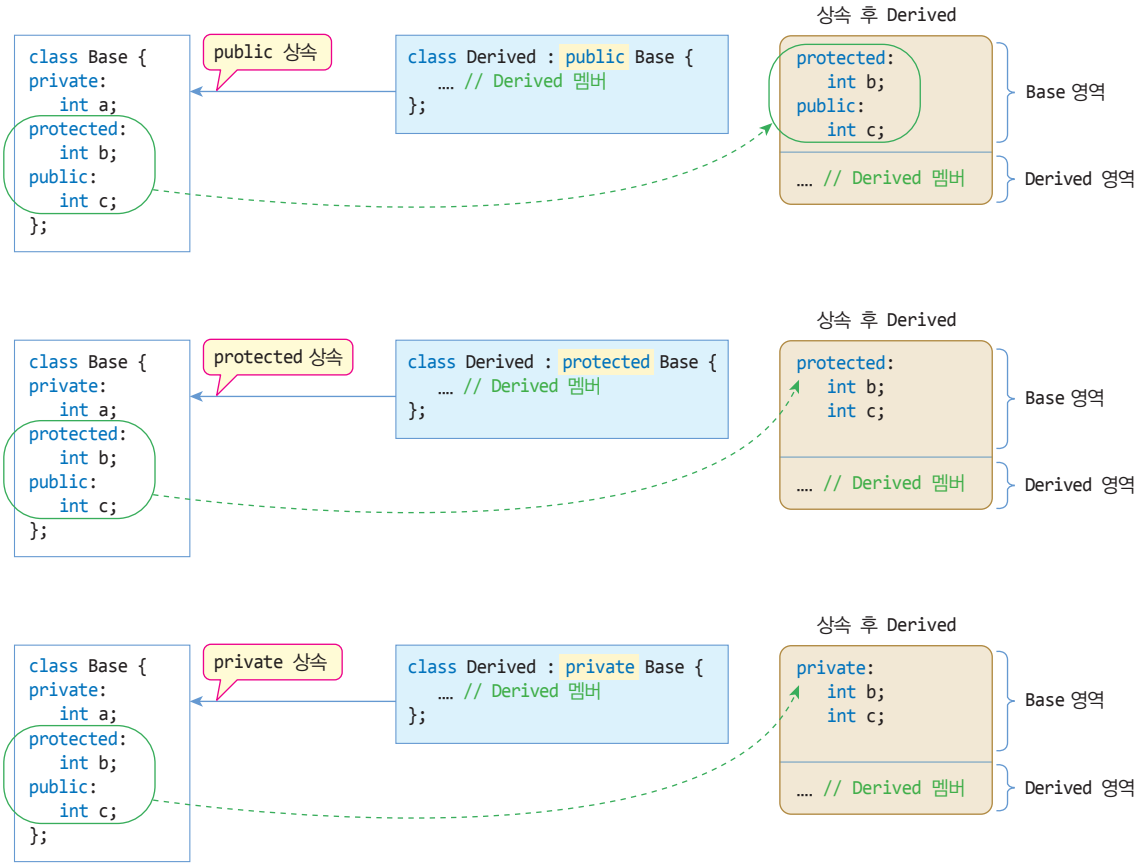
잠깐! 상속 접근 지정자가 생략되면 private 상속으로 처리

다음과 같이 상속시 접근 지정자를 생략하면 어떻게 될까?

```
class Derived : Base { ... }; // 접근 지정자 생략
```

접근 지정자가 생략되면 private 상속으로 자동 처리된다.

```
class Derived : private Base { ... };
```



[그림 1-1] 기본 클래스를 public, protected, private으로 상속받는 경우

예제 1-1 private 상속 사례

다음에서 컴파일 오류가 발생하는 부분을 찾아라.

private 상속

```
1  #include <iostream>
2  using namespace std;
3
4  class Base {
5     int a;
6  protected:
7     void setA(int a) { this->a = a; }
8  public:
9     void showA() { cout << a; }
10 };
11
12 class Derived : private Base {
13    int b;
14 protected:
15    void setB(int b) { this->b = b; }
16 public:
17    void showB() { cout << b; }
18 };
19
20 int main() {
21    Derived x;
22    x.a = 5;       // ①
23    x.setA(10);    // ②
24    x.showA();     // ③
25    x.b = 10;      // ④
26    x.setB(10);    // ⑤
27    x.showB();     // ⑥
28 }
```

Derived 클래스가 Base 클래스를 'private 상속'함에 따라 Base 클래스의 setA(), showA() 멤버는 private 속성으로 변경되어 Derived에 상속된다.

① 컴파일 오류. a는 Derived 클래스에 상속되지만 private 멤버이므로 접근 불가

②, ③ 컴파일 오류. setA(), showA()는 private 멤버로 변경되어 Derived 클래스에 상속되기 때문에 Derived 클래스 외부에서 접근 불가

④ 컴파일 오류. b는 private 멤버이기 때문에 Derived 클래스 외부에서 접근 불가

⑤ 컴파일 오류. setB()는 protected 멤버이기 때문에 main()에서 접근 불가

⑥ 정상 컴파일. showB()는 public 멤버이기 때문에 누구나 접근 가능

예제 1-2 protected 상속 사례

다음에서 컴파일 오류가 발생하는 부분을 찾아라.

protected 상속

```
1  #include <iostream>
2  using namespace std;
3
4  class Base {
5     int a;
6  protected:
7     void setA(int a) { this->a = a; }
8  public:
9     void showA() { cout << a; }
10 };
11
12 class Derived : protected Base {
13    int b;
14 protected:
15    void setB(int b) { this->b = b; }
16 public:
17    void showB() { cout << b; }
18 };
19
20 int main() {
21    Derived x;
22    x.a = 5;        // ①
23    x.setA(10);     // ②
24    x.showA();      // ③
25    x.b = 10;       // ④
26    x.setB(10);     // ⑤
27    x.showB();      // ⑥
28 }
```

Derived 클래스가 Base 클래스를 'protected 상속'함에 따라 Base 클래스의 setA(), showA() 멤버는 protected 속성으로 변경되어 Derived에 상속된다.

① 컴파일 오류. a는 Derived 클래스에 상속되지만 private 멤버이므로 접근 불가

②, ③ 컴파일 오류. setA(), showA() 멤버는 protected 멤버로 변경되어 Derived 클래스에 상속되기 때문에 Derived 클래스 외부에서 접근 불가

④ 컴파일 오류. b는 private 멤버이기 때문에 Derived 클래스 외부에서 접근 불가

⑤ 컴파일 오류. setB()는 protected 멤버이기 때문에 main() 함수에서 접근 불가

⑥ 정상 컴파일. showB()는 public 멤버이기 때문에 누구나 접근 가능

예제 1-3

상속이 중첩될 때 접근 지정 사례

다음 코드에서 컴파일 오류가 발생하는 부분을 찾아라.

```
1  #include <iostream>
2  using namespace std;
3
4  class Base {
5     int a;
6  protected:
7     void setA(int a) { this->a = a; }
8  public:
9     void showA() { cout << a; }
10 };
11
12 class Derived : private Base {
13    int b;
14 protected:
15    void setB(int b) { this->b = b; }
16 public:
17    void showB() {
18       setA(5);    // ①
19       showA();    // ②
20       cout << b;
21    }
22 };
23
24 class GrandDerived : private Derived {
25    int c;
26 protected:
27    void setAB(int x) {
28       setA(x);    // ③
29       showA();    // ④
30       setB(x);    // ⑤
31    }
32 };
```

private 상속

private 상속

Derived 클래스가 Base 클래스를 'private 상속'함에 따라 setA(), showA()는 private 속성으로 변경되어 Derived에 상속된다.

① 정상 컴파일. setA()는 Base 클래스의 protected 멤버이기 때문에 파생클래스 Derived에게 접근 허용

② 정상 컴파일. showA()는 Base 클래스의 public 멤버이기 때문에 파생 클래스 Derived에게 접근 허용

③ 컴파일 오류. `setA()`는 `private` 속성으로 변경되어 `Derived` 클래스에 상속되기 때문에 `GrandDerived` 클래스에서 접근 불가

④ 컴파일 오류. `showA()`는 `private` 속성으로 변경되어 `Derived` 클래스에 상속되기 때문에 `GrandDerived` 클래스에서 접근 불가

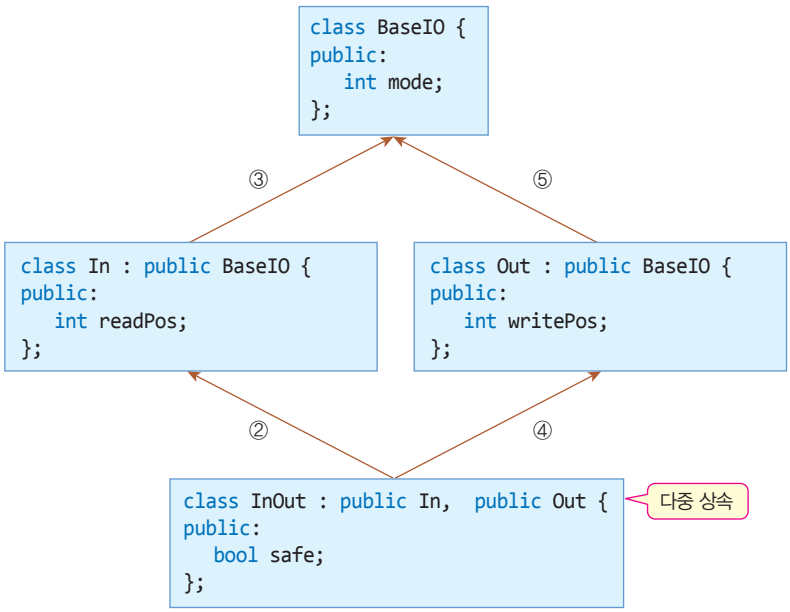
⑤ 정상 컴파일. `setB()`는 `Derived` 클래스의 `protected` 멤버이기 때문에 파생 클래스 `GrandDerived`에게 접근 허용

2. 가상 상속 [8장 관련]

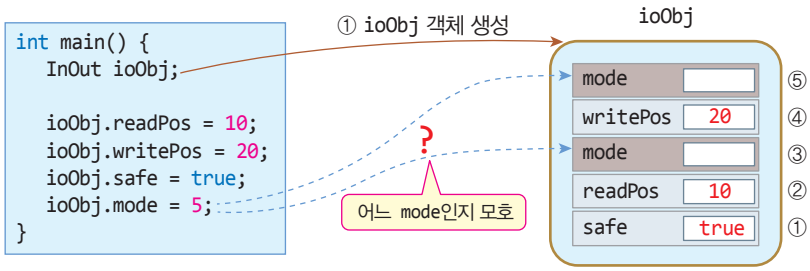
다중 상속의 문제점

다중 상속이 가진 문제점
해결 방법

다중 상속은 여러 개의 클래스를 상속받음으로써 클래스의 재사용과 코딩의 효율을 높이는 장점이 있는 반면, 보이지 않는 치명적인 문제를 내포하고 있다. 이 문제 때문에 Java 언어에서는 클래스의 다중 상속을 지원하지 않는다. 지금부터 C++의 **다중 상속이 가진 문제점**과 이 문제를 **해결**하는 **방법**을 설명해보자.



(a) 클래스 상속 관계



(b) ioObj 객체 생성 과정 및 객체 내부

[그림 2-1] 다중 상속 시 기본 클래스 멤버 접근에 모호한(ambiguous) 문제가 발생하는 경우

[그림 2-1]을 보자. BaseIO, In, Out, InOut의 4개의 클래스는 다이아몬드 형의 상속 관계를 가지며, InOut 클래스는 In과 Out 클래스를 다중 상속받고 있다. 이 경우, [그림 2-1](b)의 main() 함수에 있는 다음 코드에서 ioObj 객체가 생성되는 과정을 알아보자.

```
InOut ioObj; // ioObj 객체 생성
```

먼저 InOut 클래스의 safe 변수 공간이 배정되고(①), 기본 클래스 In의 readPos 변수 공간이 배정된다(②). 그리고 다시 BaseIO 클래스의 mode 변수 공간이 배정된다(③). InOut 클래스는 Out 클래스를 상속받기 때문에 다시 Out 클래스의 writePos 변수 공간이 배정되고(④), Out의 기본 클래스 BaseIO의 mode 변수 공간이 배정되면(⑤) ioObj 객체를 위한 공간 구성이 완료된다. 이 결과는 [그림 2-1](b)의 오른쪽과 같으며, mode 공간이 2개 생성되어 있는 것을 볼 수 있다.

ioObj 객체가 생성된 후, 다음과 같이 ioObj의 멤버를 정상적으로 접근할 수 있다.

```
ioObj.readPos = 10;
ioObj.writePos = 20;
ioObj.safe = true;
```

그러나 다음과 같이 mode 변수를 접근하면 컴파일러는 오류를 발생시킨다.

오류
```
ioObj.mode = 5; // 컴파일 오류
```

이것은 ioObj 객체에 mode가 2개 있기 때문이며, 컴파일 오류 메시지는 다음과 같다.

```
int BaseIO::mode
error : 'mode' 액세스가 모호합니다.
```

컴파일러
중복 상속

ioObj 객체 내에 2개의 mode 변수가 있기 때문에, **컴파일러**는 그 중 어떤 mode인지 판단할 수 없다는 뜻이다. 다이아몬드 형의 다중 상속 구조는 기본 클래스의 멤버가 **중복 상속**되어 객체 속에 존재하는 상황을 초래하여 컴파일 오류를 발생시키게 된다.

InOut 클래스 내에서 mode를 접근할 때도 마찬가지로 컴파일 오류가 발생한다.

오류
```
class InOut : public In, public Out {
public:
    bool safe;
    void setMode() { mode = 3; } // mode에 대한 모호성으로 인해 컴파일 오류 발생
};
```

가상 상속

●가상 상속 선언

virtual
가상 상속

다중 상속에서 생기는 멤버 중복 생성 문제를 해결하려면, 파생 클래스를 선언할 때 기본 클래스 앞에 virtual 키워드를 이용하여 **가상 상속**을 선언하면 된다. 다중 상속에 문제가 발생한 In 클래스와 Out 클래스를 다음과 같이 가상 상속으로 선언해 보자.

```
class In : virtual public BaseIO { // In 클래스는 BaseIO 클래스를 가상 상속함
 ...
};
class Out : virtual public BaseIO { // Out 클래스는 BaseIO 클래스를 가상 상속함
 ...
};
```

가상 기본 클래스
virtual base class

이때, In과 Out은 '**가상 기본 클래스**(virtual base class) BaseIO를 상속 받는다'라고 하거나 BaseIO를 '가상 상속 받는다'라고 한다. virtual 키워드는 컴파일러에게 파생 클래스의 객체가 생성될 때 기본 클래스의 멤버 공간을 오직 한 번만 할당하고, 이미 할당되어 있다면 그 공간을 공유하도록 지시한다. 그러므로 가상 기본 클래스의 멤버 공간은 오직 한 번만 생성한다.

●가상 상속으로 다중 상속의 문제 해결

모호성

[그림 2-2]는 [그림 2-1]에서 발생하는 **모호성**을 기본 클래스를 가상 상속받음으로써 해결하는 코드를 보여준다. 이 과정을 설명해보자.

다음과 같이 ioObj 객체가 생성될 때,

```
InOut ioObj;
```

InOut 클래스의 safe 변수 공간이 배정되고(①), In의 readPos 변수 공간이 배정된다(②). 그리고 아직 mode 변수 공간이 할당된 적이 없기 때문에 BaseIO 클래스의 mode 변수 공간이 배정된다(③). 다시 Out 클래스의 writePos 변수 공간이 배정된다(④).

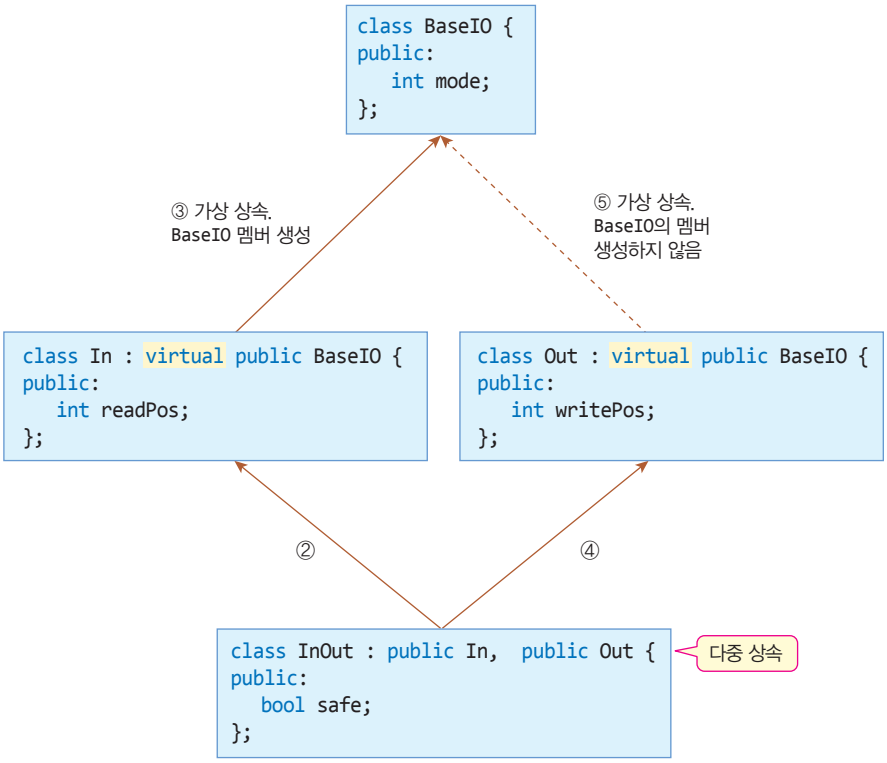
가상 상속

마지막으로 ⑤의 과정에서, BaseIO의 mode 멤버 공간이 이미 ioObj 객체에 할당되어 있기 때문에, mode 변수 공간을 추가로 만들지 않는다. Out은 BaseIO 클래스와 상속 관계는 유지하되, BaseIO의 멤버 공간을 생성하지 않고 이미 할당된 공간을 공유한다. 이것이 바로 **가상 상속**이다. 이제, mode 변수 공간이 하나만 생성되어 있기 때

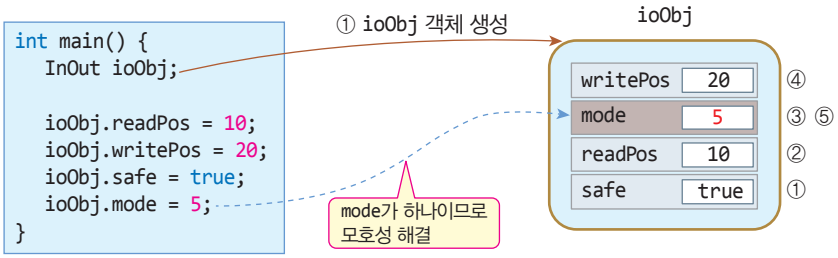
모호성

문에 mode 변수에 대한 **모호성**은 없다. 다음 코드는 컴파일 오류를 발생시키지 않는다.

```
ioObj.mode = 5; // 모호성 없음. 컴파일 오류 발생하지 않음
```



(a) 기본 클래스를 가상 상속 받는 클래스 상속 관계



(b) 가상 기본 클래스를 가진 경우, ioObj 객체 생성 과정 및 객체 내부

[그림 2-2] 기본 클래스를 가상 상속 받음으로써 다중 상속의 모호성 해결

잠깐! **가상 상속 사례 : C++ 표준 라이브러리**

기본 클래스를 가상 상속받는 경우는 C++ 표준 클래스 라이브러리에도 종종 볼 수 있다. 대표적으로 ios 클래스를 istream과 ostream이 상속받고, iostream이 istream과 ostream을 다중 상속받는 경우로서 istream과 ostream이 ios를 다음과 같이 가상 상속받아 다중 상속의 모호성 문제를 해결한다.

```
class istream : virtual public ios { ... };
class ostream : virtual public ios { ... };
class iostream : public istream, public ostream { ... };
```

CHECK TIME

1 다음 프로그램은 상속으로 인한 모호성을 가지고 있다.

```
class Person {
public:
   int id;
};
class Student : public Person { };
class Worker : public Person { };
class StudentWorker : public Student, public Worker {
   public: void getId() { return id; }
};
```

(1) 어떤 라인에서 모호성으로 인해 컴파일 오류가 발생하는가?
(2) 모호성이 없도록 수정하라.

2 다중 상속의 모호성은 virtual 키워드를 이용한 가상 상속으로 해결한다. 모호성은 어떤 시점에서 검사되는가? 컴파일 타임인가, 런타임인가?

정답

1. (1) StudentWorker 클래스의 다음 멤버에서 id 변수를 사용할 때 모호성으로 인해 컴파일 오류가 발생한다.

```
public: void getId() { return id; }
```

(2) 모호성을 해소하기 위해 다음과 같이 가상 상속으로 선언한다.

```
class Student : virtual public Person { };
class Worker : virtual public Person { };
```

2. 컴파일 타임

3. auto와 람다식 [10장 관련]

auto를 이용한 쉬운 변수 선언

auto

auto 키워드는 C++11 표준부터 의미가 수정되어, 변수 선언문으로부터 변수의 타입을 추론하여 결정하도록 지시한다. auto는 복잡한 형식의 변수 선언을 간소하게 해주며, 타입 선언의 오타나 번거로움을 줄일 수 있게 한다. auto를 사용하여 변수를 선언하는 사례를 보자.

●기본 사례

다음은 auto를 이용하여 변수 pi를 실수 3.14로 초기화하는 선언문이다.

```
auto pi = 3.14; // 3.14가 실수이므로 pi는 double 타입으로 선언됨
```

컴파일러는 3.14로부터 추론한 결과 pi를 double 타입으로 결정한다. auto를 사용하는 다른 예를 들면 다음과 같다.

```
auto n = 3; // 3이 정수이므로 n을 int로 선언
auto *p = &n; // 변수 p는 int* 타입으로 자동 선언
```

또한 다음과 같이 참조 변수의 선언에도 auto를 사용할 수 있다.

```
int n = 10;
int & ref = n; // ref는 int에 대한 참조 변수로 선언
auto ref2 = ref; // ref2는 int& 타입의 변수로 자동 선언
```

●함수의 리턴 타입으로부터 추론하여 변수 타입 선언

다음 코드를 보자.

```
int square(int x) { return x*x; }
...
auto ret = square(3); // 변수 ret는 int 타입으로 자동 선언
```

컴파일러는 square()의 리턴 타입이 int이므로 변수 ret의 타입을 int로 추론한다.

square() 함수가 double을 리턴하는 함수로 수정되면 변수 ret의 타입도 double 타입으로 자동 결정되는데, auto는 코드의 수정에 따른 오류 가능성을 낮추는 효과가 있다.

● STL 템플릿에 활용

auto를 vector, map 등의 템플릿에 사용하면 복잡한 변수 선언을 간소화할 수 있다. 예를 들어보자. 다음은 정수 1, 2, 3, 4, 5를 저장하는 벡터 v를 선언한다.

```
vector<int> v = { 1,2,3,4,5 }; // 벡터 v에 5개의 원소 1,2,3,4,5 삽입
```

iterator를 이용하여 벡터 v의 원소를 출력하려면 보통 다음 코드를 작성한다.

```
vector<int>::iterator it; // 복잡한 모양의 it 변수 선언
for (it = v.begin(); it != v.end(); it++)
    cout << *it << endl;
```

이 코드는 auto를 이용하여 변수 it를 선언하면 다음과 같이 간략히 작성할 수 있다.

```
for (auto it = v.begin(); it != v.end(); it++) // 변수 it는 vector<int>::iterator
    cout << *it << endl;                       // 타입으로 추론됨
```

예제 3-1 **auto를 이용한 변수 선언**

auto를 사용하여 변수의 타입을 선언하는 다양한 사례를 보인다.

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int square(int x) { return x*x; }
6
7  int main() {
8     // 기본 타입 선언에 auto 활용
9     auto c = 'a'; // c는 char 타입으로 결정
10    auto pi = 3.14; // pi은 double 타입으로 결정
11    auto ten = 10; // ten은 int 타입으로 결정
12    auto *p = &ten; // 변수 p는 int* 타입으로 결정
13    cout << c << " " << pi << " " << ten << " " << *p << endl;
14
15    // 함수의 리턴 타입으로 추론
16    auto ret = square(3); // square() 함수의 리턴 타입이 int 이므로 ret는 int로 결정
17    cout << *p << " " << ret << endl;
18
19    vector<int> v = { 1,2,3,4,5 }; // 벡터 v에 5개의 원소, 1,2,3,4,5 삽입
```

동일한 코드

```
20    vector<int>::iterator it;
21    for (it = v.begin(); it != v.end(); it++)
22       cout << *it << " "; // 1 2 3 4 5 출력
23    cout << endl;
24
25    // 템플릿에 auto를 사용하여 복잡한 선언의 간소화
26    for (auto it = v.begin(); it != v.end(); it++)
27       cout << *it << " "; // 1 2 3 4 5 출력
28 }
```

● 실행 결과

```
a 3.14 10 10
10 9
1 2 3 4 5
1 2 3 4 5
```

c, pi, ten, *p의 값 출력

*p, ret의 값 출력

벡터 v 출력

벡터 v 출력

람다 대수와 람다식

C++11부터 도입된 람다는 람다 대수(lambda calculus)에서 유래한다. 람다 대수에서 람다식(lambda expression)은 수학의 함수를 단순하게 표현하는 방법이다. 다음은 x, y의 합을 계산하는 수학 함수 f를 보여준다.

```
f(x, y) = x + y // x, y의 합을 구하는 수학의 함수
```

이를 수학의 람다식으로 바꾸면, 다음과 같이 이름을 빼고 간소하게 표현한다.

```
(x, y) -> x + y
```

이름 없는 함수
람다식

수학에서 **이름 없는 함수**를 **람다식**이라고 부른다. 그리고 다음과 같이 괄호와 함께 x, y에 대입될 값을 지정하면 람다식의 계산이 이루어진다. 이 람다식의 계산 결과는 5이다.

```
((x, y) -> x + y)(2, 3)
= 2 + 3
= 5
```

1930년대에 수학(mathematics)에 도입된 람다대수와 람다식은, 1960년대 중반 프로그래밍 언어에 익명의 함수 형태로 도입되었다. 몇 년 전부터는 Java, C#, 자바스

크립트, 파이선 등 여러 언어들이 람다식을 도입하였고, C++는 C++11 표준부터 지원한다.

C++에서 람다식 선언

이름 없는 익명 함수
람다식
람다 함수

이제 프로그래밍의 세계로 들어와서 C++의 람다식에 대해 알아보자. 프로그래밍의 세계에서 람다는 **이름 없는 익명 함수**(anonymous function)로, **람다식**(lambda expression) 혹은 **람다 함수**로 불린다. 람다식의 기본 구조는 [그림 3-1]과 같이 캡쳐 리스트, 매개 변수 리스트, 리턴 타입, 함수 바디의 4부분으로 구성된다.

(a) 람다식의 구조

```
[](int x, int y) { cout << x + y; };          // 매개 변수 x, y의 합을 출력하는 람다 작성
[](int x, int y) -> int { return x + y; };    // 매개 변수 x, y의 합을 리턴하는 람다 작성
[](int x, int y) { cout << x + y; } (2, 3);   // x에 2, y에 3을 대입하여 코드 실행. 5 출력
```

(b) 람다식 작성 및 호출 사례

[그림 3-1] 람다식의 기본 구조와 사례

● 캡쳐 리스트

캡쳐 리스트는 이름에서도 느껴지듯, 람다식의 외부에 선언된 변수(지역변수, 전역변수) 목록으로, 람다식에서 사용하고자 할 때 나열하며 〈표 3-1〉과 같이 여러 표현들이 있다.

〈표 3-1〉 캡쳐 리스트에 작성 가능한 요소

표현	의미	표현	의미
[x]	변수 x의 값 활용	[&x]	참조 변수 x 활용
[=]	모든 변수의 값 활용	[&]	모든 참조 변수 활용

● 매개 변수 리스트

보통 함수의 매개 변수 리스트와 같다.

● 함수 바디

람다식이 호출될 때 실행되는 코드로서, 함수를 작성하는 방법과 동일하다.

간단한 람다식 만들기

예제 3-2를 통해 직관적으로 람다식을 선언하고 호출하는 방법을 알아보자.

예제 3-2 **매개 변수 x, y의 합을 출력하는 람다식 만들기**

매개 변수 x, y의 합을 출력하는 람다식은 다음과 같이 작성할 수 있지만,

```
[](int x, int y) { cout << x + y; }; // x, y의 합을 출력하는 람다식
```

다음 예제 코드와 같이 x에 2, y에 3을 전달하면 람다식이 바로 실행된다.

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5     [](int x, int y) { cout << "합은 " << x + y; } (2, 3); // 람다식 실행. 5 출력
6  }
```

● 실행 결과

```
합은 5
```

auto로 람다식 저장 및 호출

람다식의 형식은 컴파일러에만 알려져 있기 때문에, 개발자가 람다식을 저장하는 변수를 직접 선언할 수 없다. auto를 이용하면 람다식을 저장하는 변수를 쉽게 선언할 수 있다. 예제 3-3을 통해 알아보자.

예제 3-3 **auto로 람다식 다루기**

auto를 이용하여 변수 love에 람다식을 저장하고, love()의 모양으로 람다식을 호출하는 사례이다. 매개 변수 a, b에 문자열이 전달되는 것도 눈 여겨 보라.

람다식을 변수 love에 저장

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  int main() {
6     auto love = [](string a, string b) {
7        cout << a << "보다 " <<  b << "가 좋아" << endl;
8     };
9
10    love("돈", "너");  // 람다식 호출
11    love("냉면", "만두"); // 람다식 호출
12 }
```

● 실행 결과

```
돈보다 너가 좋아
냉면보다 만두가 좋아
```

캡쳐 리스트와 리턴 타입을 가지는 람다식 만들기

람다식은, 캡쳐 리스트를 이용하여 주변의 non-static 변수들(지역 변수나 멤버 변수)에 대해 값을 복사하여 받거나 참조를 활용할 수 있다. 예제 3-4는 람다식이 캡쳐 리스트를 통해 외부 변수의 값을 전달받는 사례이며, 예제 3-5는 캡쳐 리스트를 통해 지역 변수의 참조를 전달받고 참조를 이용하여 지역 변수를 변경하는 사례를 보인다.

예제 3-4 반지름이 r인 원의 면적을 리턴하는 람다식 만들기

다음은 지역 변수(pi)의 값을 받고, 매개 변수 r을 이용하여 반지름 값을 받아, 원의 면적을 계산하는 람다식 사례이다. 람다식은 원의 면적을 리턴한다.

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5     double pi = 3.14; // 지역 변수
6
7     auto calc = [pi](int r) -> double { return pi*r*r; }; // 반지름이 r인 원 면적을
                                                              리턴하는 람다식 작성
8     cout << "면적은 " << calc(3); // 람다식 호출. 28.26 출력
9  }
```

3.14

지역 변수 pi의 값을 람다식에서 활용한다는 의미

● 실행 결과

```
면적은 28.26
```

예제 3-5 캡쳐 리스트에 참조 활용. 합을 외부에 저장하는 람다식 만들기

지역 변수에 대한 참조를 캡쳐 리스트로 받아 활용하는 사례이다. 캡쳐 리스트로부터 참조를 받게 되면, 참조 변수를 통해 외부에 값을 전달할 수 있다. 이 예제는 캡쳐 리스트를 통해 지역 변수 sum에 대한 참조를 받고 합한 결과를 지역 변수 sum에 저장한다. 이런 방식으로 람다식은 지역 변수에 값을 전달할 수 있다.

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5     int sum = 0; // 지역 변수
6
7     [&sum](int x, int y) { sum = x + y; } (2, 3); // 합 5를 지역 변수 sum에 저장
8
9     cout << "합은 " << sum;
10 }
```

람다 내의 sum은 지역 변수 sum의 참조이므로 지역 변수 sum을 바꾸게 됨

● 실행 결과

```
합은 5
```

STL 템플릿에 람다식 활용

람다식은 STL 템플릿을 사용하여 프로그램을 작성할 때 더욱 유용하다. 먼저 예제 3-6을 통해 람다식 없이 STL 알고리즘을 활용하는 경우를 알아보고, 예제 3-7에서 람다식을 사용하여 좀더 간편하게 STL 알고리즘을 사용하는 방법을 알아보자.

예제 3-6 STL 함수 for_each()를 이용하여 벡터의 모든 원소 출력

for_each()는 컨테이너의 각 원소를 검색하는 STL 알고리즘 함수이다. for_each()는 컨테이너의 각 원소에 대해 3번째 매개 변수에 주어진 함수를 호출한다. 이 예제는 정수 1, 2, 3, 4, 5를 담고 있는 벡터의 각 원소를 출력하는 코드이다.

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm> // for_each() 알고리즘 함수를 사용하기 위함
4  using namespace std;
5
6  void print(int n) {
7     cout << n << " ";
8  }
9
10 int main() {
11    vector<int> v = { 1, 2, 3, 4, 5 };
12
13    // for_each()는 벡터 v의 첫번째 원소(v.begin())부터 끝(v.end())까지 검색하면서,
14    // 각 원소에 대해 3번째 매개 변수인 print(int n) 호출. 매개 변수 n에 각 원소 값 전달
15    for_each(v.begin(), v.end(), print);
16 }
```

print()를 v의 원소 개수만큼 호출하고 각 원소를 매개 변수 n에 전달

● 실행 결과

```
1 2 3 4 5
```

예제 3-6에서 print() 함수는 for_each()에서만 호출되고 다른 곳에서는 사용되지 않는다. 이런 경우 예제 3-7과 같이 print() 함수를 람다식으로 바꾸면 전체 코드가 깔끔해진다.

예제 3-7 STL 함수 for_each()와 람다식을 이용하여 벡터의 모든 원소 출력

예제 3-6의 print() 함수를 익명의 람다식으로 수정하라.

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm> // for_each() 알고리즘 함수를 사용하기 위함
4  using namespace std;
5
6  int main() {
7     vector<int> v = { 1, 2, 3, 4, 5 };
8
9     // for_each()는 벡터 v의 첫번째 원소부터 끝까지 검색하면서,
10    // 각 원소에 대해 3번째 매개 변수인 람다식 호출. 매개 변수 n에 각 원소 값 전달
11    for_each(v.begin(), v.end(), [](int n) { cout << n << " "; });
12 }
```

● 실행 결과

```
1 2 3 4 5
```

C++에서 람다식의 의미

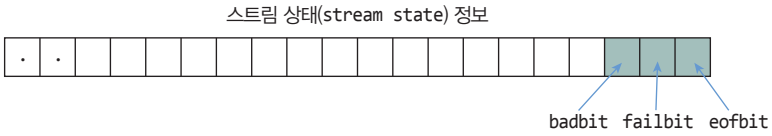
C++ 프로그램 작성 시 람다가 꼭 필요한 기능은 아니다. 다만 간단하고 짧게 최적화된 코드를 작성하는데 도움이 된다. 특별히 람다가 유용한 경우는 다음과 같다.

- 한 번만 호출하고 재사용하지 않기 때문에 함수에 이름을 붙일 필요가 없는 경우
- STL 알고리즘 함수의 매개 변수에 연산 코드를 넘기는 경우, 연산 코드를 익명의 람다식으로 작성(예제 3-7)

4. 파일 입출력 스트림 상태 검사 [12장 관련]

스트림의 상태
오류 정보

파일 입출력 스트림은 **스트림의 상태**(stream state)를 저장하는 멤버 변수를 두고, 입출력이 진행되는 동안 발생한 **오류**(error) **정보**를 유지한다. 스트림 상태를 나타내는 멤버 변수는 [그림 4-1]과 같고, 〈표 4-1〉은 각 오류 비트 상수를 보여준다. 응용프로그램에서는 스트림의 상태를 검사하기 위해 〈표 4-2〉의 함수를 이용하면 된다. 예제 4-1은 파일 입출력 시 오류가 발생하는 경우 스트림의 상태를 검사하는 몇 가지 사례를 보여준다.



[그림 4-1] 스트림 상태를 저장하는 멤버 변수의 각 오류 비트들

〈표 4-1〉 스트림의 상태를 나타내는 각 비트

비트	설명
eofbit	파일의 끝을 만났을 때 1로 세팅
failbit	정수를 입력받고자 하였으나 문자열이 입력되는 등 포맷 오류나, 쓰기 금지된 곳에 쓰기를 시행하는 등 전반적인 I/O 실패 시에 1로 세팅
badbit	스트림이나 데이터가 손상되는 수준의 진단되지 않는 문제가 발생한 경우나 유효하지 않는 입출력 명령이 주어졌을 때 1로 세팅

〈표 4-2〉 스트림의 상태를 검사하는 멤버 함수

멤버 함수	설명
eof()	파일의 끝을 만났을 때(eofbit=1) true 리턴
fail()	failbit나 badbit가 1로 세팅되었을 때 true 리턴
bad()	badbit이 1로 세팅되었을 때 true 리턴
good()	스트림이 정상적(모든 비트가 0)일 때 true 리턴
clear()	스트림 상태 변수를 0으로 지움

예제 4-1

스트림 상태 검사

파일 입출력 스트림의 상태를 검사하는 사례를 들어보자. 존재하지 않는 파일을 파일 입력 스트림으로 열 때와, 존재하는 파일을 열 때 스트림 상태를 검사해 본다. 그리고 파일 읽기가 끝나는 시점에서 다시 스트림의 상태를 검사해본다.

```
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4
5  void showStreamState(ios& stream) {
6     cout << "eof() " << stream.eof() << endl;
7     cout << "fail() " << stream.fail() << endl;
8  cout << "bad() " << stream.bad() << endl;
9     cout << "good() " << stream.good() << endl;
10 }
11
12 int main() {
13    const char* noExistFile = "c:\\temp\\noexist.txt"; // 존재하지 않는 파일명
14    const char* existFile = "c:\\temp\\student.txt"; // 존재하는 파일명
15
16    ifstream fin(noExistFile); // 존재하지 않는 파일 열기
17    if(!fin) { // 열기 실패 검사
18       cout << noExistFile << " 열기 오류" << endl;
19       showStreamState(fin); // 스트림 상태 출력
20
21       cout << existFile << " 파일 열기" << endl;
22       fin.open(existFile);
23       showStreamState(fin); // 스트림 상태 출력
24    }
25
26    // 스트림을 끝까지 읽고 화면에 출력
27    int c;
28    while((c=fin.get()) != EOF)
29       cout.put((char)c);
30
31    cout << endl;
32    showStreamState(fin); // 스트림 상태 출력
33
34    fin.close();
35 }
```

존재하지 않는 파일을 열 때, 스트림의 상태가 어떻게 변하는지 알기 위한 열기 시도

정상적인 파일을 열 때, 스트림의 상태가 어떠한지 보기 위한 열기 시도

EOF를 만났을 때, 스트림의 상태 출력

● 실행 결과

```
c:\temp\noexist.txt 열기 오류
eof() 0
fail() 1
bad() 0
good() 0
c:\temp\student.txt 파일 열기
eof() 0
fail() 0
bad() 0
good() 1
kitae
20131111
computer
eof() 1
fail() 1
bad() 0
good() 0
```

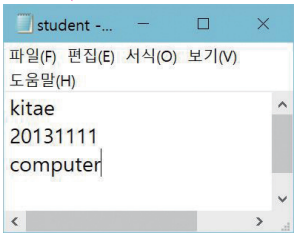
라인 19 실행 결과

라인 23 실행 결과

라인 28~29 실행 결과

라인 32 실행 결과

c:₩temp₩student.txt 파일이 있어야 함



파일의 끝
운영체제

잠깐! 파일의 마지막에 파일의 끝을 나타내는 EOF(-1) 데이터가 있을까?

파일의 끝이란 구체적으로 무엇일까? **파일의 끝**은 파일의 마지막 데이터가 기록된 지점을 지난 물리적인 끝 지점이다. 텍스트 파일의 경우 역사적으로 DOS 운영체제 시절에는 파일의 마지막에 EOF 문자(0x1A)를 자동으로 기록하였다. 그리고 읽는 도중 0x1A를 만나면 파일의 끝으로 인식하였다. 그러나 이 방식은 DOS 시절부터 비교적 과거의 마이크로소프트의 운영체제에서만 통용되어 왔으며, 맥(Mac OS)이나 유닉스(Unix) 등 운영체제마다 조금씩 다르게 나타난다.

현재 윈도우 운영체제에서는 텍스트 파일의 끝에 0x1A를 자동으로 기록하지 않는다. 메모장으로 파일을 작성해보면 파일의 끝에 0x1A가 자동으로 삽입되지 않는 것을 볼 수 있다. 그러나 과거에 만들어진 파일의 경우 0x1A가 있을 수도 있다. 한편, 바이너리 파일의 경우에는 파일의 끝을 나타내는 특정 데이터가 정의된 적이 없다.

텍스트 파일이든 바이너리 파일이든, 텍스트 I/O든 바이너리 I/O든, 파일의 끝을 인식하는 작업은 **운영체제**(OS)의 몫이다. 입출력 라이브러리는 운영체제의 도움을 받아, API 함수를 통해 응용프로그램에게 파일의 끝을 알린다. 운영체제의 파일 시스템은 각 파일의 크기 정보를 가지고 있기 때문에, 파일 읽기가 진행되는 동안 파일 포인터가 파일의 끝에 도달하는지 알 수 있다. 파일 포인터가 파일의 마지막 데이터를 넘어 접근하게 되면, 이때 파일의 끝에 도달하였음을 기억해두고, 입출력 라이브러리가 요청할 때 이 사실을 알려준다.

5. 파일 입출력 임의 접근 [12장 관련]

순차 접근과 임의 접근

순차 접근
임의 접근
디폴트는 순차 접근

C++ 입출력 라이브러리의 get(), put(), read(), write() 입출력 함수들은, 마지막으로 읽은 다음 위치에서 읽고, 마지막으로 기록한 다음 위치에 기록한다. 우리는 지금까지 이런 입출력의 특징을 자연스럽게 받아들이며, 파일의 처음부터 끝까지 순차적으로 읽거나 순차적으로 파일에 기록하는 **순차 접근**(sequential file access) 방식만 다루었다. C++ 입출력 시스템은 순차 접근뿐 아니라, 파일 내에 임의의 위치로 옮겨 다니면서 읽거나 쓰는 **임의 접근**(random file access) 방식도 지원한다. C++ 파일 입출력 시스템의 **디폴트는 순차 접근** 방식이다.

파일 포인터

파일 포인터

파일은 연속된 바이트의 집합이다. C++ 파일 입출력 스트림은 파일 내의 읽고 쓸 바이트의 위치를 가리키는 **파일 포인터**(file pointer)라는 특별한 마크(mark)를 두고 있다. C++에서 파일 포인터는 다음 2가지 종류가 있으며, 열려진 파일(혹은 스트림)마다 별도의 파일 포인터가 유지된다.

get pointer
put pointer

- get pointer – 파일 내의 읽기 지점을 가리키는 파일 포인터
- put pointer – 파일 내의 쓰기 지점을 가리키는 파일 포인터

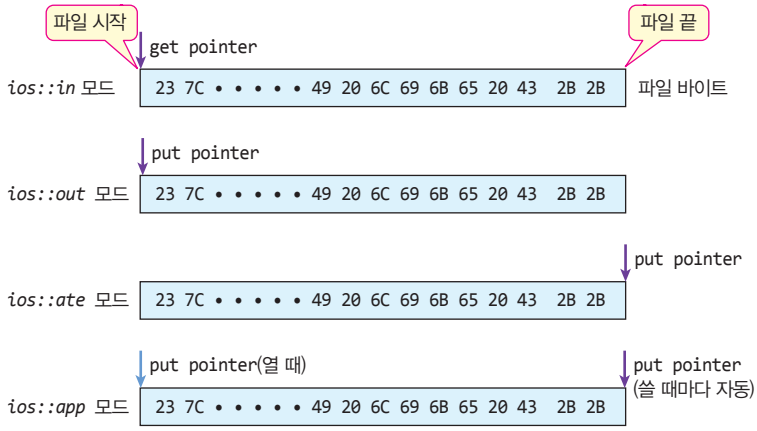
ifstream이나 읽기 모드(ios::in)로 열려진 fstream은 get pointer만을 가진다. get(), read(), getline(), >> 등의 읽기 연산은 get pointer가 가리키는 지점에서 읽고, 읽은 바이트 수만큼 get pointer를 전진시킨다. 반대로 ofstream이나 쓰기 모드(ios::out)로 열려진 fstream은 put pointer 만을 가진다. put(), write(), << 등의 쓰기 연산은 put pointer가 가리키는 위치에 데이터를 쓰고 쓴 바이트 수만큼 put pointer를 전진시킨다.

파일 모드와 파일 포인터

파일 포인터의 위치
파일 모드

파일 포인터의 위치는 **파일 모드**에 따라 다르게 설정된다. [그림 5-1]은 파일 모드에 따른 파일 포인터의 위치를 보여준다. 읽기 모드(ios::in)로 파일을 열면 get pointer는 파일의 맨 처음 위치를 가리키며, 쓰기 모드(ios::out)로 파일을 열면 put pointer가 파일의 맨 처음 위치를 가리킨다. ios::ate 모드로 파일을 열면 put pointer를 파일의 맨 끝, 즉 EOF 위치를 가리키며 여기서부터 쓰기가 시작된다. ios::app 모드로 파일을

열면 put pointer는 파일의 처음 위치를 가리키지만, 파일 쓰기를 하면 자동으로 맨 끝으로 put pointer를 옮겨서 파일의 끝에 쓰기를 수행한다.



[그림 5-1] 파일 모드와 파일 포인터

임의 접근 방법

파일 포인터
임의의 위치로 이동
절대적 위치
상대적 위치

임의 접근 방법의 핵심은 **파일 포인터**의 위치를 **임의의 위치로 이동**시키는데 있다. 파일 포인터는 읽기나 쓰기 이후 항상 전진하지만, 다음 멤버 함수를 이용하면 언제든지 파일 포인터를 임의의 위치로 이동시킬 수 있다. 다음 멤버 함수에서 streampos는 파일 포인터의 **절대적 위치** 값으로 실제 long 타입이며, streamoff는 파일 포인터의 **상대적 위치** 값으로 역시 long 타입이다. seekbase에 사용되는 상수는 〈표 5-1〉과 같다.

istream& seekg(streampos pos)
  정수 값으로 주어진 절대 위치 pos로 get pointer를 옮김
istream& seekg(streamoff offset, ios::seekdir seekbase)
  seekbase를 기준으로 offset 만큼 떨어진 위치로 get pointer를 옮김
ostream& seekp(streampos pos)
  정수 값으로 주어진 절대 위치 pos로 put pointer를 옮김
ostream& seekp(streamoff offset, ios::seekdir seekbase)
  seekbase를 기준으로 offset 만큼 떨어진 위치로 put pointer를 옮김
streampos tellg()
  입력 스트림의 현재 get pointer의 값 리턴
streampos tellp()
  출력 스트림의 현재 put pointer의 값 리턴

〈표 5-1〉 seekbase로 사용되는 ios::seekdir 타입의 상수

seekbase	설명
ios::beg	파일의 처음 위치를 기준으로 파일 포인터를 움직인다.
ios::cur	현재 파일 포인터의 위치를 기준으로 파일 포인터를 움직인다.
ios::end	파일의 끝(EOF) 위치를 기준으로 파일 포인터를 움직인다.

인덱스 0부터 시작

파일 포인터는 **인덱스 0부터 시작**한다. seekg()와 tellg()는 입력 스트림에서만 사용하고, seekp()와 tellp()는 출력 스트림에서만 사용한다. [그림 5-2]는 seekg() 함수로 get pointer를 이동시키는 사례를 보여준다. 그림과 함께 아래의 코드 사례를 보자.

●절대 위치로 파일 포인터 옮기기

다음은 seekg(5)를 이용하여 파일의 인덱스 5(0부터 시작) 위치로 파일 포인터를 옮기고 0x49 값을 읽는 사례이다.

```
fin.seekg(5); // get pointer를 파일의 인덱스 5 위치로 옮긴다.
int n = fin.tellg(); // n은 5가 된다.
int c = fin.get(); // c=0x49
```

●상대 위치로 파일 포인터 옮기기

매개 변수를 2개 가진 seekg(), seekp() 함수는 seekbase를 기준으로 offset만큼 상대 위치로 파일 포인터를 움직인다. 사용 예를 들면 다음과 같다.

```
fin.seekg(10, ios::beg); // 파일 시작점을 기준으로 get pointer를 10바이트 전진시킨다.
fin.seekg(-1, ios::cur); // 현재 위치에서 get pointer를 한 바이트 후진시킨다.
fin.seekg(0, ios::end); // 파일의 맨 끝(EOF 위치)으로 get pointer를 움직인다.
                        // 여기서부터 읽을 데이터는 없다.
```

●파일의 맨 마지막 문자를 읽는 경우

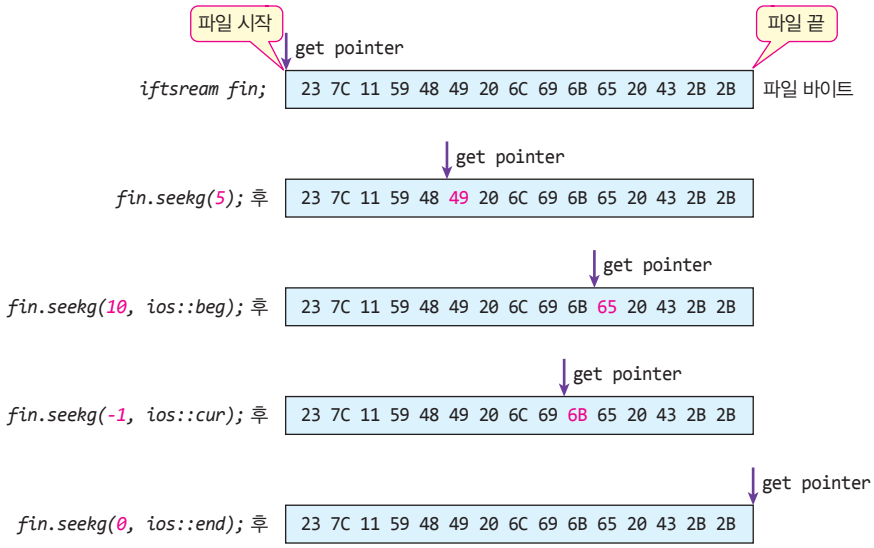
```
fin.seekg(-1, ios::end); // get pointer를 파일의 마지막 문자 위치로 보낸다.
int c = fin.get(); // 파일의 마지막 문자 0x2B를 c에 읽어 온다.
                   // get() 실행 후 get pointer는 EOF를 가리킴
```

● 10바이트 간격으로 문자를 읽고자 하는 경우

```
int c;
while((c=fin.get()) != EOF) { // get()에 의해 get pointer는 1바이트 전진 이동
    fin.seekg(9, ios::cur);  // 나머지 9바이트를 건너뛰어 get pointer 전진 이동
}
```

● 텍스트 파일을 거꾸로 화면에 출력하는 경우

```
fin.seekg(0, ios::end); // get pointer를 파일의 EOF 위치로 이동
int fileSize = fin.tellg(); // get pointer 값이 바로 파일의 크기임
for(int i=0; i<fileSize; i++) {
    fin.seekg(fileSize-1-i, ios::beg); // 파일의 처음을 기준으로 get pointer 이동
    int c = fin.get();
    cout << (char)c;
}
```



[그림 5-2] seekg()에 의한 get pointer의 이동 사례

파일 포인터를 이용하여 파일 크기 알아내기

파일 포인터를 파일의 맨 끝으로 이동
파일 크기

C++ 표준 입출력 라이브러리에서는 파일 크기를 아는 방법을 제공하지 않는다. 그렇지만 파일 포인터를 이용하여 파일 크기를 쉽게 알아 낼 수 있다. **파일 포인터를 파일의 맨 끝으로 이동**시킨 뒤 파일 포인터의 위치를 알아내면 그 값이 바로 **파일 크기**이다.

리눅스나 유닉스의 C++, 윈도우의 Visual C++ 등 많은 컴파일러들이 파일 크기나 파일 속성을 다루는 그들만의 비표준 파일 입출력 라이브러리를 제공하지만, 상호 호환성은 없다. 예제 5-1은 파일 크기를 알아내는 코드를 보여주며, 텍스트 파일이든 바이너리 파일이든 상관없이 작동한다.

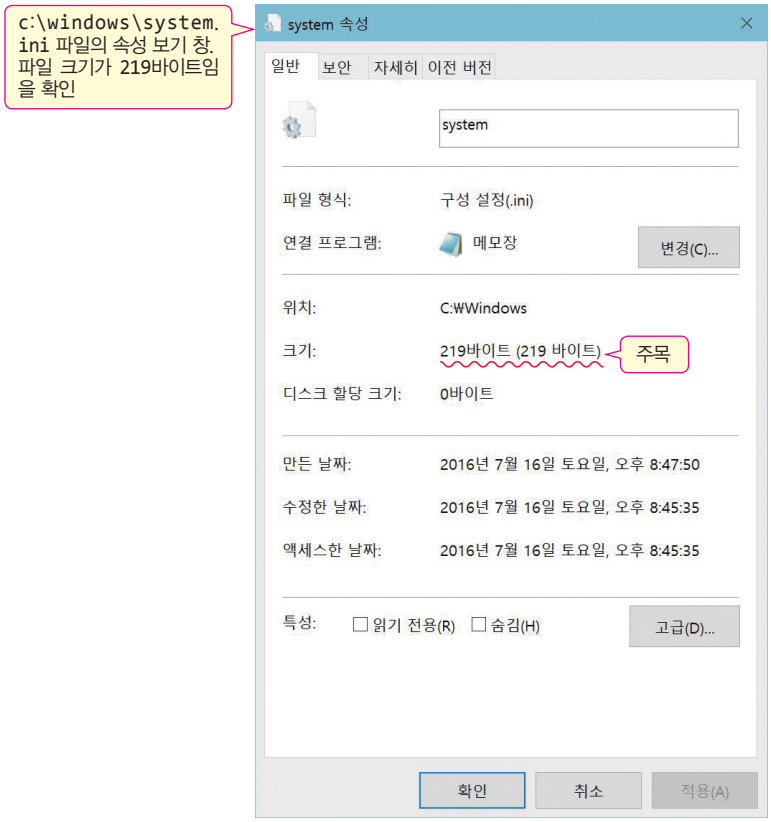
예제 5-1 **파일 크기 알아내기**

c:\windows\system.ini 파일의 크기는 몇 바이트인지 알아내어 출력하라.

```
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4
5  long getFileSize(ifstream& fin) {
6     fin.seekg(0, ios::end); // get pointer를 파일의 맨 끝으로 옮김
7     long length = fin.tellg(); // get pointer의 위치를 알아냄
8     return length; // length는 파일의 크기와 동일
9  }
10
11 int main() {
12    const char* file = "c:\\windows\\system.ini";
13
14    ifstream fin(file);
15    if(!fin) { // 열기 실패 검사
16       cout << file << " 열기 오류" << endl;
17       return 0;
18    }
19    cout << file << "의 크기는 " << getFileSize(fin);
20    fin.close();
21 }
```

● 실행 결과

```
c:\windows\system.ini의 크기는 219
```


c:\windows\system.ini 파일의 속성 보기 창. 파일 크기가 219바이트임을 확인
system 속성
일반
보안
자세히
이전 버전
system
파일 형식:
구성 설정(.ini)
연결 프로그램:
메모장
변경(C)...
위치:
C:₩Windows
크기:
219바이트 (219 바이트)
주목
디스크 할당 크기:
0바이트
만든 날짜:
2016년 7월 16일 토요일, 오후 8:47:50
수정한 날짜:
2016년 7월 16일 토요일, 오후 8:45:35
액세스한 날짜:
2016년 7월 16일 토요일, 오후 8:45:35
특성:
읽기 전용(R)
숨김(H)
고급(D)...
확인
취소
적용(A)